

Patent Draft Information

1. Full name, nationality and address of applicant(s):

Full Name	Nationality	Address
Vishwakarma Institute of Technology	Indian	666, Upper Indiranagar, Bibwewadi, Pune, Maharashtra, India – 411 037
Vishwakarma University	Indian	Survey No 2, 3, 4, Kondhwa Main Rd, Laxmi Nagar, Betal Nagar, Kondhwa, Pune, Maharashtra, India - 411048

2. Full name (including middle name), nationality, address, mail id, and phone number of inventor(s):

Full Name (Including middle name)	Nationality	VIT Address (Start with full dept. name followed by full institute name) (For ex.)	Mail ID	Phone No.
Soham Moholkar	Indian	Department of Computer Science and Engineering (Data Science), Vishwakarma Institute of Technology, Pune	soham.moholkar241@vit.edu	
Shashank Kakad	Indian	Department of Computer Science and Engineering (Data Science), Vishwakarma Institute of Technology, Pune	Shashank.Kakad241@vit.edu	
Omkar Kakade	Indian	Department of Computer Science and Engineering (Data Science), Vishwakarma Institute of Technology, Pune	Omkar.kakade24@vit.edu	
Yash Pardeshi	Indian	Department of Computer Science and Engineering (Data Science), Vishwakarma Institute of Technology, Pune	Yash.pardeshi24@vit.edu	
Rupali Mahajan	Indian	Department of Computer Science and Engineering (Data Science), Vishwakarma Institute of Technology, Pune	rupali.mahajan@vit.edu	

Signatures of inventors to be inserted before filing

Soham Moholkar	Shashank Kakad	Omkar Kakade	Yash Pardeshi

Rupali Mahajan			

3. Title of the invention:

INVARIANT Based Security Reasoning for Software Repositories

4. Technical field of the invention:

The invention relates to computer engineering, application security, static program analysis, data flow analysis, repository intelligence, and computer implemented security review systems.

5. Prior art:

Existing code assistants and conventional security scanners can generate or review code, identify known patterns, or report isolated findings. They may not construct an application level model of users, resources, actions, data flows, and authorization controls before assessing whether a security rule holds across a complete code path. The proposed system builds that model and applies an INVARIANT algorithm that derives and evaluates security rules against evidence from the repository. Specific patent and publication references shall be added after a formal prior art search.

6. Objective(s) of Invention:

To identify the security rules that must remain true in a software application, trace the code and data paths relevant to each rule, distinguish verified controls from possible gaps, and return review items with evidence that a developer can inspect.

7. Synopsis:

The system receives a software repository and scans source files, routes, APIs, data models, authorization checks, and external inputs. It constructs an application model and uses the INVARIANT algorithm to identify sensitive resources and actions, define security conditions for those resources and actions, trace relevant code paths, and record evidence for each condition. A decision engine returns a review item as verified, at risk, or requiring human review, together with the relevant repository locations and suggested remediation context.

8. Brief description of drawings (if any):

Figure 1 illustrates the Invaria system architecture. Figure 2 illustrates the INVARIANT algorithm workflow.

Figure 1 shows the repository, business-context, and policy inputs; secure ingestion and code extraction; the security graph and INVARIANT engine; evidence verification and decision ledger; supporting security guidance, optional model review, review feedback; and persistence, workspace, and report delivery. Figure 2 shows repository capture, application-model construction, sensitive-path identification, invariant

derivation, path tracing, evidence collection, evidence gating, candidate recording, and human review.

9. Detailed description of the invention:

The repository scanner identifies entry points, request handlers, service functions, database operations, data models, authorization controls, configuration values, and externally controlled identifiers. The application model builder associates these elements with actors, resources, actions, and reachable paths. The model may use language-model or retrieval assistance to explain unfamiliar code, but the system records a finding only when repository evidence supports the corresponding decision.

For each candidate invariant, the algorithm creates a rule tuple comprising an actor, a resource, an action, and one or more required conditions. A representative rule requires that a requested action on a protected resource must be associated with an authenticated actor and a matching authorization, ownership, tenant, state, or policy check. The code and control flow analyzer traces the request from its entry point through service logic to a sensitive operation. The data flow analyzer determines whether a request controlled identifier or value reaches that operation. The evidence verifier compares the observed checks with the required conditions and records the supporting or missing evidence in the ledger.

10. Best method of performance of the invention:

In a preferred embodiment, a team connects a repository and identifies the application's relevant environment or framework. The system scans the repository, forms the application model, and selects sensitive resources and actions, such as customer records, transfers, refunds, tenant files, administrative actions, credentials, or payments. For each selected item, the INVARIANT algorithm creates a condition of the form: an actor may perform an action on a resource only when the required authentication, authorization, ownership, tenant, state, or policy condition holds.

For example, when an API receives an account identifier and retrieves a customer record, the algorithm traces the identifier from the route to the data access operation. It then seeks evidence that the requester is authenticated and that the customer record belongs to the requester or the requester's tenant. If the path reaches the record without a matching check, the decision engine records the route, code locations, affected invariant, impact explanation, and a suggested review action.

11. CLAIMS:

1. A computer implemented security review system comprising a repository scanner configured to obtain a software repository; an application model builder configured to identify entry points, code paths, data flows, resources, actions, and security controls; an INVARIANT engine configured to derive a security condition for a resource or action; a code and data flow analyzer configured to trace a path associated with the security condition; an evidence verifier; an evidence ledger; and a decision engine configured to produce a security review item associated with repository evidence.
2. The system of claim 1, wherein the security condition comprises an actor, a resource, an action, and at least one required condition selected from authentication, authorization, ownership, tenant separation, state transition validity, credential protection, or policy compliance.
3. The system of claim 1, wherein the code and data flow analyzer identifies whether an externally controlled identifier or value reaches a protected resource or sensitive operation without evidence of the required condition.
4. A computer implemented method comprising receiving a software repository; constructing an application model; identifying a protected resource or sensitive action; deriving an invariant comprising a required security condition; tracing a code and data path associated with the invariant; determining whether repository evidence supports the required security condition; recording a verified result, potential violation, or human review result; and outputting the result with repository locations and remediation context.
5. The method of claim 4, wherein a language model provides contextual assistance for interpreting source code, and wherein the evidence verifier requires code, control flow, data flow, configuration, or policy

evidence before the decision engine records a result as verified.

12. Inventive step of your invention:

The inventive step is the INVARIANT algorithm. Rather than starting with a fixed vulnerability signature or a general code prompt, the algorithm derives application specific security rules from a repository model, identifies the actors, resources, actions, and policy conditions involved, and evaluates each rule along the relevant control and data paths. The same rule is connected to the evidence that supports a verified result, a potential violation, or an unresolved result. This produces a security review that remains tied to the application's own behavior and repository locations.

13. Industrial application:

The invention is useful for software teams developing web applications, software as a service products, financial technology products, healthcare systems, e-commerce platforms, enterprise applications, and AI assisted software projects that handle user data, money, credentials, or privileged actions.

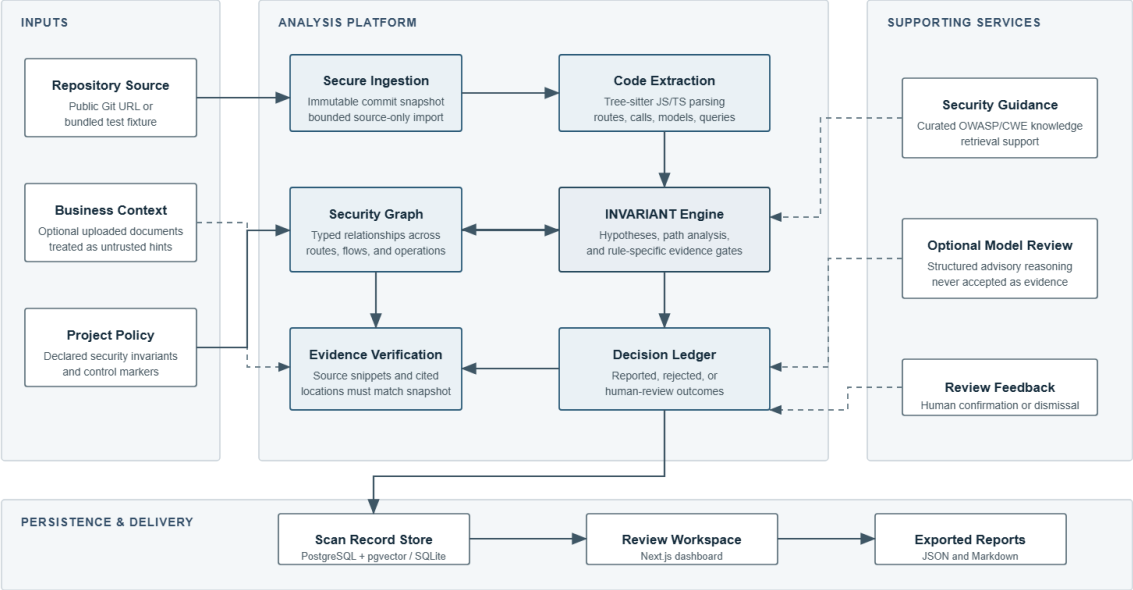
14. Abstract:

A computer implemented security review system receives a software repository and constructs an application model containing repository entry points, code paths, data flows, resources, actions, and security controls. An INVARIANT algorithm derives a security condition for a selected resource or action, traces the corresponding code and data paths, and evaluates whether authentication, authorization, ownership, tenant, state, or policy conditions are evidenced along the path. The system records a verified result, a potential violation, or a result requiring human review, and outputs an evidence ledger containing repository locations, the relevant path, the affected condition, and remediation context.

15. Drawing

INVARIA — SYSTEM ARCHITECTURE

Evidence-first static application-security review

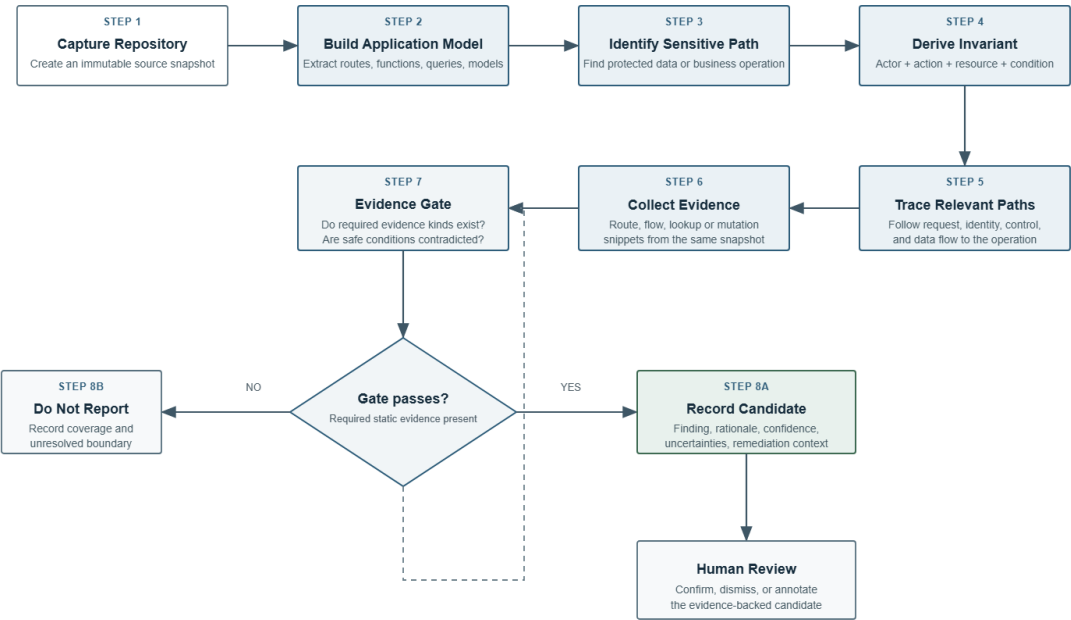


Invarya architecture overview · Static analysis, evidence gating, and human review

Figure 1

INVARIANT — ALGORITHM WORKFLOW

Invariant-based Networked Verification and Attack-path Reasoning



Feedback and new project policy can refine future hypotheses; source evidence remains the decision boundary.

INVARIANT prevents model prose or contextual documents from substituting for required repository evidence.

Figure 2